

Class 6: R Functions

Kai Christman (A17522126)

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

Q1A: Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11. [1 pt]

A first function

Here we will create a function to add some numbers. Let's call it `add()` (this is similar to the function `sum()`).

Input arguments can either be **required** or **optional**. The latter have fallback default values, such as setting `y` to a default value that can be changed in later arguments.

```
add <- function(x, y=1000) {  
  x + y  
}
```

Can we use the new function?

```
add(10, 100)
```

```
[1] 110
```

Q1B: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`. [1 pt]

Version 2 of our function

This function will be able to take a single input **vector** or two inputs, similar to the first iteration.

```
add <- function(x, y=0) {  
  sum(x, y)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4, 7, 10) )
```

```
[1] 21
```

Q1C: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the... (dots) argument or the base R function `sum()`. [2 pts]

Version 3 of our function

This function returns anything that the user inputs in a total sum.

```
add <- function(x, y=0, ...) {  
  sum(x, y, ...)  
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using `return()` function call.

```
add <- function(x, y=0, ...) {
  return(sum(x, y, ...))
  cat("Is it break time yet?\n")
}
```

```
add(10, 100)
```

```
[1] 110
```

#3 A generate_dna() function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size = 5, replace = TRUE)
```

```
[1] 5 4 5 4 4
```

We can use `sample()` to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T” ...

```
sample( x = c("A", "C", "G", "T"), size = 12, replace = TRUE )
```

```
[1] "C" "T" "T" "A" "G" "A" "T" "T" "C" "C" "G" "A"
```

Q2: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user.

Q2A: Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

This function will take an **input** which is the length of the desired sequence. As a result, a random sequence of that length will be produced.

```
generate_dna <- function(len = 10) {
  sample( x = c("A", "C", "G", "T"), size = len, replace = TRUE )
}
```

```
generate_dna(len=100)
```

```
[1] "A" "C" "A" "T" "T" "G" "A" "C" "G" "T" "C" "A" "T" "A" "A" "G" "C" "C"
[19] "C" "C" "C" "T" "G" "G" "A" "G" "G" "A" "C" "A" "G" "G" "A" "C" "G" "G"
[37] "T" "C" "T" "T" "A" "T" "G" "G" "A" "G" "C" "T" "T" "C" "C" "C" "C" "A"
[55] "A" "T" "C" "C" "A" "A" "C" "C" "A" "A" "A" "A" "A" "T" "C" "T" "C" "T"
[73] "A" "T" "C" "G" "A" "G" "T" "T" "A" "T" "G" "C" "G" "A" "A" "C" "T" "A"
[91] "T" "C" "G" "A" "C" "A" "T" "T" "G" "G"
```

Q2B: Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
generate_dna <- function(len, single.element = TRUE) {
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE )

  if(single.element) {
    paste(ans, collapse = "")
  }
  else
    return(ans)
}

generate_dna(len = 5, single.element = TRUE)
```

```
[1] "TAATG"
```

(Functions that could be useful are `paste()`, `if()`, `cat()`, and `return()`)

Q2C: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

```
generate_dna <- function(len, single.element = TRUE) {
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE )

  if(single.element) {
    ans <- paste(ans, collapse = "")
  }
}
```

```
## Format as FASTA with an ID line
cat(paste( ">len",len, "\n", sep= "" ))
cat(ans)

}

generate_dna(len = 44)
```

```
>len44
TTGTCCACCTACAGCTTAAGGGGTGGCCCCGTTAGAAATAAGGT
```

Write a generate_protein() function

Q3: Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return “WQRTAG”. Your function should: • Use the single-letter code for all 20 standard amino acids and no other letters (see earlier list at the beginning of this handout) and include clear comments that explain each step of your function. [2 pts]

Using the sequence of 20 Amino Acids, return only the amount of amino acids that are requested

```
generate_protein <- function(len, single.element = TRUE) {
  acids = c("A", "C", "D", "E", "F", "G", "H", "I", "K", "L", "M", "N", "P", "Q", "R", "S", "T", "V", "W", "Y")

  ans <- sample(x = acids, size = len, replace = TRUE)

  if(single.element) {
    paste(ans, collapse = "")
  }

}

generate_protein(20)
```

```
[1] "PTVEVCMNDTPVPILNNEEW"
```

Generate random protein sequences of length 6 to 13

Q4: Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp. For example: id.6 NTYHEC id.7 MVRSLAW id.8 ...

```
for(l in 6:13) {  
  cat(">", l, "\n", sep="")  
  cat( generate_protein(l), "\n" )  
}
```

```
>6  
HDWYDG  
>7  
SSRAPCY  
>8  
ICKYFMII  
>9  
MSIPSGIHK  
>10  
RGTEGEVSVE  
>11  
QYYYYARNCTG  
>12  
DFTHCWAKECVR  
>13  
MIERVHRHWAMDR
```

Are these peptides “unique in nature”?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched). For each of your 8 peptides (lengths 6 through 13), inspect the top hit and record: • the % identity of the best hit, and • the % query coverage of the best hit. We will define a peptide as “unique in nature” (in the specific sense used in this lab) if no match exists with 100% coverage AND 100% identity to a sequence already in nr. Note

that this is a conservative definition — a short exact match may still exist as a sub-string of a longer protein, and will typically be reported by BLASTp with 100% identity but less than 100% coverage of the subject (which we are not using here). In your quarto report create a fill in table along the lines of the following: [2 pts]

Length	Identity	Coverage	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	100	89	Y
10	90	100	Y
11	81	100	Y
12	75	100	Y
13	91	85	Y

a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

Peptides start to look unique in nature once you reach the 8 amino acid mark. This was when our class started to notice discrepancy on who was generating unique for non-unique sequences.

b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^n for a peptide of length n) and to the size of the known protein universe. [1 pt]

Very short random peptides are almost always found in nr given that there is less variability in the sequence of amino acids. Smaller sequences of amino acids can be more “universal” while longer sequences of amino acids likely contribute to specific proteins or may contribute to nothing at all. The known protein universe is finite while the sequence of amino acids is infinite, meaning that there will almost certainly be a random sequence of amino acids produced that is not within the known protein universe.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Given that a majority of unique yes/no discrepancy begins after an 8 amino acid length sequence, I would guess that this is the minimum length used. If the immune MHC class II molecules use smaller sequences, it increases the likelihood that they are NOT unique. This could potentially be harmful as viral bacteria and harmful pathogens may be able to disguise themselves given their genomic identity. This would cause the MHC class II molecules with the help of T cells to identify them as “self” peptides and allowing them to stay alive.